

Appendix S1. User guide to the *cocktailr* R package

Contents

Installation	4
1. Data requirements	4
2. Building the Cocktail tree: <code>cocktail_cluster()</code>	6
2.1. Purpose	6
2.2. Key arguments	7
2.3. Returned components	7
2.4. Example: basic clustering (wide format) with relative cover output	8
2.5. Example: long-format input with relative cover output	9
3. Visualizing the dendrogram: <code>cocktail_plot()</code>	9
3.1. Purpose	9
3.2. Interface	10
3.3. Examples	10
4. Selecting parent clusters at a ϕ cut: <code>clusters_at_cut()</code>	12
4.1. Purpose	12
4.2. Usage	12
4.3. Example	12
5. Selecting diagnostic species groups: <code>select_clusters()</code>	12
5.1. Purpose	12
5.2. Usage	13
5.3. Examples	13
6. Relationships among selected groups: <code>cluster_dist()</code>	14
6.1. Purpose	14
6.2. Containment distance	15
6.3. Phi distance	15

6.4. Usage	16
6.5. Example	16
7. Species sets and diagnostic lists: <code>species_in_clusters()</code>	18
7.1. Purpose	18
7.2. Usage	18
7.3. Examples	18
8. Listing plots in clusters or combinations of clusters: <code>releves_in_clusters()</code>	19
8.1. Purpose	19
8.2. Usage	20
8.3. Examples	20
9. Finding clusters containing species: <code>clusters_with_species()</code>	21
9.1. Purpose	21
9.2. Usage	21
9.3. Examples	22
10. Assigning plots (relevés) to candidate vegetation units: <code>assign_releves()</code>	22
10.1. Purpose	22
10.2. Interface	22
10.3. Strategies	23
10.4. Example	24
11. Typical analysis workflow (dune dataset)	25
11.1. Step 1: Prepare vegetation matrix	25
11.2. Step 2: Run Cocktail clustering	25
11.3. Step 3: Plot the Cocktail dendrogram	25
11.4. Step 4: Define candidate clusters	26
11.5. Step 5: Species lists (cluster species and ϕ -ranked diagnostics)	27
11.6. Step 6: Relationships among selected clusters	28
11.7. Step 7: Assign plots to candidate vegetation units	29
12. Reproducibility notes	29

This appendix describes the main workflow and functions of the **cocktailr** package, which implements a fast and reproducible version of the Cocktail algorithm for clustering vegetation-plot data.

The typical workflow is:

1. Prepare vegetation data either as a **plots** $\times$ **species** matrix or as a **long-format vegetation table** with plot, species and value columns.
2. Run **cocktail_cluster()** to build the Cocktail species-group hierarchy and, if needed, compute **species-cluster ϕ profiles**.
3. Visualize and inspect the Cocktail hierarchy with **cocktail_plot()** and extract broader parent clusters at a chosen ϕ cut with **clusters_at_cut()**.
4. Select candidate diagnostic species groups with **select_clusters()**, using filtering criteria, alternative score methods and non-nesting rules.
5. Explore relationships among selected Cocktail groups with **cluster_dist()**, using either plot-membership containment or direct co-membership ϕ .
6. Inspect the selected groups with helper functions: use **species_in_clusters()** to list constituting or ϕ -ranked diagnostic species, **releves_in_clusters()** to list plots satisfying cluster membership rules, and **clusters_with_species()** to find clusters containing target species.
7. Assign plots (relevés) to candidate vegetation units with **assign_releves()**, using count-, cover-, ϕ - or cover-weighted ϕ -based assignment strategies.

Below we describe each function in detail with reproducible examples.

Installation

The development version of `cocktailr` can be installed from GitHub using:

```
install.packages("remotes")
remotes::install_github("dvynokur/cocktailr")
```

The version of `cocktailr` used in the manuscript was archived as GitHub release v0.1.1 and can be installed using:

```
remotes::install_github("dvynokur/cocktailr@v0.1.1")
```

After installation, load the package with:

```
library(cocktailr)
```

1. Data requirements

`cocktailr` accepts vegetation data either as a **plots × species** table (**wide format**) or as a **long-format** table with plot, species and value columns.

In the **wide format**, rows are plots (relevés), columns are species, and cell values are typically **cover/abundance**. In the **long format**, each row represents one plot–species record together with its associated value.

Cocktail clustering itself is based on **presence/absence** only: internally, values > 0 are treated as presence (1), and values ≤ 0 or NA as absence (0). Cover values therefore do **not** influence the merge topology.

Cover values can still be used downstream if requested:

- If `plot_values = "rel_cover"` in `cocktail_cluster()`, the function stores **relative cover** for each plot and cluster in `Plot.cluster`, defined as (sum of the cluster species covers in the plot) / (total cover in the plot), with values set to 0 for plots that do not meet the cluster membership threshold (m) and for plots with total cover = 0.

Basic checks on the cover scale are performed inside `cocktail_cluster()`:

- if values appear **binary (0/1)** or contain **non-numeric codes** (e.g. +, r, 2a), the function warns and falls back to binary plot-membership output (`plot_values = "binary"`).
- if values look **ordinal** (small integer scales), the function warns but proceeds; percentage cover is recommended.

Minimum requirements:

Wide format

- plots in rows, species in columns
- `colnames(vegmatrix) = species names (required)`
- `rownames(vegmatrix) = plot names (strongly recommended)`
- species columns with no presences (all zeros after binarization) are dropped internally before clustering

Long format

- one column with plot IDs
- one column with species names or species IDs
- one column with numeric abundance/cover values
- duplicate plot–species records are aggregated internally
- empty plots are removed internally

Example toy matrix (wide format):

```
vm <- matrix(c(1, 0, 5,
               0, 10, 0,
               2, 5, 0,
               0, 1, 3),
             nrow = 4, byrow = TRUE,
             dimnames = list(
               paste0("plot", 1:4),
               c("sp1", "sp2", "sp3")
             ))
```

```
vm
#>      sp1 sp2 sp3
#> plot1  1  0  5
#> plot2  0 10  0
#> plot3  2  5  0
#> plot4  0  1  3
```

Example toy matrix (long format):

```
vm_long <- data.frame(
  plot      = c("plot1", "plot1", "plot2", "plot3", "plot3", "plot4", "plot4"),
  species   = c("sp1", "sp3", "sp2", "sp1", "sp2", "sp2", "sp3"),
  value     = c(1, 5, 10, 2, 5, 1, 3)
)
```

```

vm_long
#>   plot species value
#> 1 plot1      sp1      1
#> 2 plot1      sp3      5
#> 3 plot2      sp2     10
#> 4 plot3      sp1      2
#> 5 plot3      sp2      5
#> 6 plot4      sp2      1
#> 7 plot4      sp3      3

```

2. Building the Cocktail tree: `cocktail_cluster()`

2.1. Purpose

`cocktail_cluster()` performs **Cocktail agglomeration** on vegetation data supplied either as a **plots** × **species** table (wide format) or as a **long-format** table with plot, species and value columns.

For two binary presence vectors representing two species, ϕ is computed from the 2×2 contingency table counts:

- a : number of plots where both species occur
- b : plots where species 1 occurs and species 2 does not
- c : plots where species 2 occurs and species 1 does not
- d : plots where neither species occurs

$$\phi = \frac{ad - bc}{\sqrt{(a + c)(b + d)(a + b)(c + d)}}.$$

Internally, the function:

- **Binarizes** input for clustering: values > 0 are set to 1; values ≤ 0 or NA are set to 0.
- Drops **empty species columns** (all zeros) before clustering.
- For **long-format** input, aggregates duplicate plot–species records and removes empty plots.
- Computes ϕ each round from a sparse crossproduct for speed and exactness.
- Uses a **deterministic tie order**: ties in maximum ϕ are resolved in the fixed order corresponding to the lower-triangular scan order used by R’s distance objects.
- Stores `Plot.cluster` as a **sparse dgMatrix**, containing either binary membership (`plot_values = "binary"`) or relative cover (`plot_values = "rel_cover"`).
- Optionally computes **species–cluster ϕ profiles** (`Species.cluster.phi`), a species × clusters matrix of ϕ between species presence and cluster membership.

2.2. Key arguments

```
cocktail_cluster(  
  vegmatrix,  
  progress          = TRUE,  
  plot_values       = c("binary", "rel_cover"),  
  species_cluster_phi = TRUE,  
  input_format      = c("wide", "long"),  
  long              = list(plot = "plot",  
                           species = "species",  
                           value = "value"),  
  save_vegmatrix    = TRUE  
)
```

- **vegmatrix**: vegetation data supplied either as a plots $\times$ species matrix or data.frame (input_format = "wide") or as a long-format data frame with plot, species and value columns (input_format = "long").
- **plot_values**:
 - "binary" (default): Plot.cluster stores 0/1 plot membership per cluster.
 - "rel_cover": Plot.cluster stores relative cover per plot and cluster (set to 0 outside cluster membership).
- **species_cluster_phi**:
 - TRUE (default): compute Species.cluster.phi (species $\times$ clusters ϕ profiles).
 - FALSE: do not compute ϕ profiles.
- **input_format**:
 - "wide" (default): interpret vegmatrix as a plots $\times$ species table.
 - "long": interpret vegmatrix as a long-format table.
- **long**: named list specifying the column names used for long-format input, with elements plot, species, and value.
- **save_vegmatrix**:
 - TRUE (default): store the internally used, cleaned and aligned vegetation matrix (vegmatrix) as a sparse dgCMatrix (plots $\times$ species), which can be reused in downstream functions (e.g. assign_relevés()).
 - FALSE: do not store it, reducing memory usage.

2.3. Returned components

The result `res` is a list of class "cocktail" with:

- `Cluster.species`: clusters $\times$ species matrix (0/1 membership of species in each cluster).
- `Cluster.info`: clusters $\times$ 2 integer matrix with columns `k` (number of species constituting the cluster) and `m` (cluster membership threshold).
- `Cluster.merged`: clusters $\times$ 2 integer matrix describing the merge topology; each row corresponds to one cluster, and the two columns store its two children (negative values indicate original species, positive values indicate previously formed clusters).
- `Cluster.height`: ϕ at each merge (cluster height).
- `Plot.cluster`: plots $\times$ clusters sparse matrix (binary membership or relative cover).
- `Species.cluster.phi` (optional): species $\times$ clusters ϕ profiles.
- `vegmatrix` (optional): cleaned and aligned vegetation matrix stored in the result if `save_vegmatrix = TRUE` as a sparse `dgCMatrix` (plots $\times$ species).
- `species, plots`: names aligned to the cleaned dataset.

2.4. Example: basic clustering (wide format) with relative cover output

```
set.seed(1)
vm <- matrix(sample(c(0,0,0,1,2,5,10), 80, replace = TRUE),
              nrow = 10,
              dimnames = list(
                paste0("plot", 1:10),
                paste0("sp", 1:8)
              ))

res <- cocktail_cluster(
  vegmatrix          = vm,
  plot_values        = "rel_cover",
  species_cluster_phi = TRUE,
  save_vegmatrix     = TRUE
)
#> |

names(res)
#> [1] "Cluster.species"      "Cluster.info"
#> [3] "Plot.cluster"         "Cluster.merged"
#> [5] "Cluster.height"       "Species.cluster.phi"
#> [7] "species"              "plots"
#> [9] "vegmatrix"
```


2.5. Example: long-format input with relative cover output

```
res_long <- cocktail_cluster(  
  vegmatrix      = vm_long,  
  plot_values    = "rel_cover",  
  species_cluster_phi = TRUE,  
  input_format   = "long",  
  long           = list(plot = "plot",  
                        species = "species",  
                        value = "value")  
)  
#> |  
  
class(res_long$Plot.cluster)  
#> [1] "dgCMatrix"  
#> attr(,"package")  
#> [1] "Matrix"  
res_long$Plot.cluster[1:min(5, nrow(res_long$Plot.cluster)),  
                      1:min(5, ncol(res_long$Plot.cluster)),  
                      drop = FALSE]  
#> 4 x 2 sparse Matrix of class "dgCMatrix"  
#>  
#> plot1 1.0000000 1  
#> plot2 .          1  
#> plot3 0.2857143 1  
#> plot4 0.7500000 1  
res_long$vegmatrix  
#> 4 x 3 sparse Matrix of class "dgCMatrix"  
#>      sp1 sp3 sp2  
#> plot1 1   5   .  
#> plot2 .   .  10  
#> plot3 2   .   5  
#> plot4 .   3   1
```

3. Visualizing the dendrogram: `cocktail_plot()`

3.1. Purpose

`cocktail_plot()` draws a dendrogram-style representation of the Cocktail tree, with merge heights equal to ϕ .

- Species are shown as tips.
- Clusters are shown as successive merges.
- The y-axis shows the merge ϕ .
- Optional background bands can indicate clusters retained at a chosen ϕ cut.
- Optional highlighting of user-specified clusters or combinations of clusters.
- Output can be written to PDF/PNG or displayed on the active graphics device.

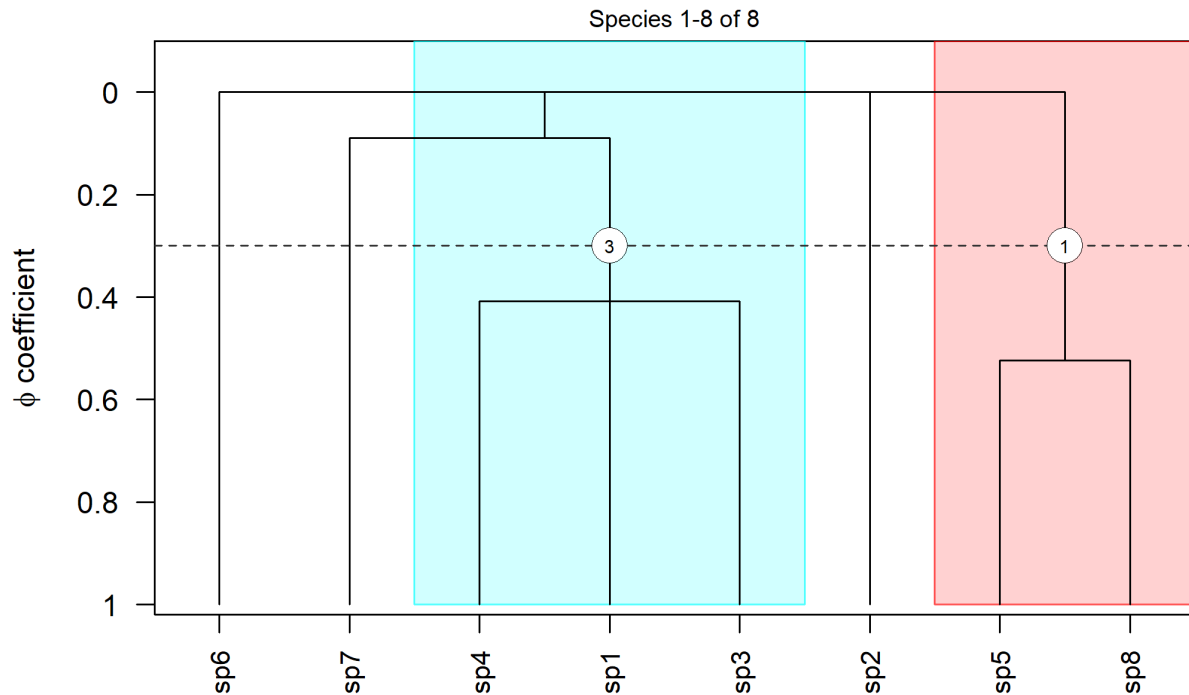
3.2. Interface

```
cocktail_plot(
  x,
  file           = NULL,
  phi_cut        = NULL,
  clusters       = NULL,
  label_clusters = FALSE,
  cex_species    = 0.3,
  cex_clusters   = 0.6,
  circle_inch    = 0.1,
  page_size      = 300,
  width_in       = NULL,
  height_in      = 10,
  alpha_fill     = 0.18,
  alpha_border   = 0.65,
  palette        = "rainbow",
  png_res        = 300
)
```

3.3. Examples

Plot to the current device with a background band at a chosen ϕ cut:

```
cocktail_plot(
  x           = res,
  file        = NULL,
  phi_cut     = 0.30,
  label_clusters = TRUE,
  cex_species = 0.9
)
```



Highlight selected clusters (vector) or combinations of clusters (list):

```
# selected clusters as a vector
cocktail_plot(x = res,
              clusters = c("c_2", "c_5"),
              label_clusters = TRUE
)

# combinations of clusters specified as a list
cocktail_plot(
  x = res,
  clusters = list(group1 = c("c_2", "c_5"), group2 = "c_7"),
  label_clusters = TRUE
)
```

4. Selecting parent clusters at a ϕ cut: `clusters_at_cut()`

4.1. Purpose

`clusters_at_cut()` identifies the parent clusters at or above a chosen ϕ threshold. These are the topmost clusters that meet the cut level, i.e. none is nested within another above-cut cluster.

4.2. Usage

```
clusters_at_cut(x, phi_cut, as_labels = TRUE)
```

- **phi_cut**: numeric ϕ cut (0–1).
- **as_labels**: return "c_<id>" labels (TRUE) or integer IDs (FALSE).

4.3. Example

```
parent_labels <- clusters_at_cut(res,
                                phi_cut = 0.30,
                                as_labels = TRUE
)
parent_labels
#> [1] "c_1" "c_3"
```

5. Selecting diagnostic species groups: `select_clusters()`

5.1. Purpose

`select_clusters()` filters and ranks Cocktail clusters and returns either cluster labels, numeric cluster IDs, or a table of selected clusters and their statistics.

By default, clusters are ranked using the operational score

$$\text{score}_c = h_c \cdot \log(k_c) \cdot \log(m_c),$$

where h_c is the merge height of cluster c , k_c is the number of species in the cluster, and m_c is the Cocktail membership threshold. This score is intended as a practical ranking criterion, not as a universal measure of diagnostic quality.

Alternative score methods are available. With `score_method = "h_logk"`, clusters are ranked by $h \cdot \log(k)$, so the membership threshold m does not affect ranking. With `score_method = "h"`, clusters are ranked only by merge height.

Candidate clusters can be filtered by merge height (`min_phi`), species-group size (`min_k`), number of member plots (`min_n`) and selection score (`min_score`). Selection can also be restricted to clusters contained within a broader cluster using `within_cluster`.

Because the Cocktail hierarchy contains nested groups, the `mode` argument controls how nested candidates are handled. With `mode = "strict"`, a selected cluster excludes both its ancestors and its descendants from further selection. With `mode = "top"`, only the hierarchically highest eligible clusters are retained.

5.2. Usage

```
select_clusters(  
  x,  
  clusters = NULL,  
  within_cluster = NULL,  
  min_phi = 0.2,  
  min_k = 1L,  
  min_n = 1L,  
  min_score = 0,  
  score_method = c("h_logk_logm", "h_logk", "h"),  
  mode = c("strict", "top"),  
  return = c("labels", "ids", "table")  
)
```

5.3. Examples

```
strong_labels <- select_clusters(  
  x = res,  
  min_phi = 0.20,  
  min_k = 1L,  
  min_n = 1L,  
  min_score = 0,  
  score_method = "h_logk_logm",  
  mode = "strict",  
  return = "labels"  
)  
  
strong_labels  
#> [1] "c_3" "c_1"
```

Score table:

```
strong_table <- select_clusters(  
  x = res,  
  min_phi = 0.20,  
  min_k = 1L,  
  min_n = 1L,  
  min_score = 0,  
  score_method = "h_logk_logm",  
  mode = "strict",  
  return = "table"  
)  
  
strong_table  
#>   cluster id      h k m n      score  
#> 1      c_3   3 0.4082483 3 3 3 0.4927348  
#> 2      c_1   1 0.5238095 2 2 6 0.2516659
```

Optional example without the membership threshold in the ranking score:

```
strong_table_h_logk <- select_clusters(  
  x = res,  
  min_phi = 0.20,  
  min_k = 1L,  
  min_n = 1L,  
  min_score = 0,  
  score_method = "h_logk",  
  mode = "strict",  
  return = "table"  
)  
  
strong_table_h_logk
```

6. Relationships among selected groups: `cluster_dist()`

6.1. Purpose

`cluster_dist()` computes pairwise distances between Cocktail clusters based on binary plot-membership vectors derived from `Plot.cluster`.

Membership is derived as:

```
membership = (Plot.cluster > 0).
```

This works for both `plot_values = "binary"` and `plot_values = "rel_cover"`, because in both cases positive values indicate plots satisfying the Cocktail membership rule of a cluster.

In principle, distances can be computed among all Cocktail clusters by setting `clusters = NULL`. In large datasets, however, this often produces a very large and difficult-to-interpret distance matrix. A more common workflow is to first select a manageable set of candidate diagnostic groups with `select_clusters()` and then examine relationships among these selected groups.

Two distance methods are available:

- `method = "containment"`: distance is based on plot-membership containment and is calculated as $1 - \max(C_{A|B}, C_{B|A})$. This is the default and is useful for detecting nested or partly nested relationships among Cocktail groups.
- `method = "phi"`: distance is calculated as $1 - \phi$, where ϕ is the phi coefficient between binary plot-membership vectors. This corresponds to symmetric direct co-membership association.

6.2. Containment distance

For two clusters A and B , let n_A and n_B be the numbers of plots satisfying their membership rules, and let n_{AB} be the number of plots satisfying both rules.

Directional containment is calculated as:

$$C_{A|B} = \frac{n_{AB}}{n_A}, \quad C_{B|A} = \frac{n_{AB}}{n_B}.$$

The similarity is the larger of the two directional containment values:

$$\text{sim}(A, B) = \max(C_{A|B}, C_{B|A}),$$

and the distance is:

$$d(A, B) = 1 - \text{sim}(A, B).$$

Containment distance ranges from 0 to 1.

6.3. Phi distance

For `method = "phi"`, the phi coefficient is computed from the 2×2 table of plot co-membership:

- a : plots where $A = 1$ and $B = 1$,
- b : plots where $A = 1$ and $B = 0$,
- c : plots where $A = 0$ and $B = 1$,
- d : plots where $A = 0$ and $B = 0$.

$$\phi = \frac{ad - bc}{\sqrt{(a + c)(b + d)(a + b)(c + d)}}.$$

Distance is then:

$$d(A, B) = 1 - \phi(A, B).$$

Since ϕ can be negative, phi distances can range from 0 to 2.

6.4. Usage

```
cluster_dist(
  x,
  clusters = NULL,
  method = c("containment", "phi"),
  return = c("dist", "matrix", "table")
)
```

6.5. Example

Select clusters first:

```
clusters_for_dist <- select_clusters(
  x = res,
  min_phi = 0.20,
  min_k = 1L,
  min_n = 1L,
  min_score = 0,
  score_method = "h_logk_logm",
  mode = "strict",
  return = "labels"
)

clusters_for_dist
#> [1] "c_3" "c_1"
```


Compute containment distance among selected clusters:

```
D <- cluster_dist(
  x = res,
  clusters = clusters_for_dist,
  method = "containment"
)

D
#>           c_1
#> c_3 0.6666667
```

Inspect the pairwise relationship table:

```
cluster_dist(
  x = res,
  clusters = clusters_for_dist,
  method = "containment",
  return = "table"
)
#>   cluster_1 cluster_2 id_1 id_2 n_1 n_2 shared_n
#> 1         c_1         c_3    1    3    6    3         1
#>   containment_1_in_2 containment_2_in_1 containment_max
#> 1           0.1666667           0.3333333           0.3333333
#>   containment_min jaccard          phi          method distance
#> 1           0.1666667    0.125 -0.3563483 containment 0.6666667
```

Cluster the selected groups. In this small toy example, only two clusters are selected. Therefore, the pairwise containment table is shown, but no dendrogram is plotted. In larger datasets with three or more selected clusters, the distance object can be passed to `hclust()`.

```
hc_clusters <- hclust(D, method = "average")

plot(
  hc_clusters,
  main = "Cluster dendrogram (containment distance)",
  cex = 0.8
)
```

For direct co-membership phi distance, use:

```
D_phi <- cluster_dist(
  x = res,
  clusters = clusters_for_dist,
```

```
method = "phi"  
)
```

7. Species sets and diagnostic lists: `species_in_clusters()`

7.1. Purpose

`species_in_clusters()` returns species sets for selected Cocktail clusters or for combinations of clusters.

- By default, it returns the **cluster-constituting species set** (from `Cluster.species`).
- If `species_cluster_phi = TRUE`, it uses `Species.cluster.phi` to report ϕ values per species and cluster, allowing ranked and ϕ -filtered diagnostic lists.

7.2. Usage

```
species_in_clusters(  
  x,  
  labels,  
  species_cluster_phi = FALSE,  
  min_phi              = NULL,  
  top_k               = NULL,  
  sort_desc           = TRUE  
)
```

7.3. Examples

Cluster-constituting species sets:

```
sp_sets <- species_in_clusters(  
  x      = res,  
  labels = c("c_2", "c_5")  
)  
str(sp_sets)  
#> List of 2  
#> $ c_2: chr [1:2] "sp1" "sp3"  
#> $ c_5: chr [1:3] "sp2" "sp5" "sp8"
```

Combinations of clusters:

```
union_sets <- species_in_clusters (
  x      = res,
  labels = list(
    group1 = c("c_1", "c_2"),
    group2 = "c_4"
  )
)
union_sets
#> $c_1_2
#> [1] "sp1" "sp3" "sp5" "sp8"
#>
#> $c_4
#> [1] "sp1" "sp3" "sp4" "sp7"
```

Diagnostic lists with ϕ (requires `species_cluster_phi = TRUE` in `cocktail_cluster()`):

```
sp_phi <- species_in_clusters (
  x              = res,
  labels         = c("c_2", "c_5"),
  species_cluster_phi = TRUE,
  min_phi       = 0.25,
  top_k         = 10
)
sp_phi$c_5
#>   species      phi
#> 1     sp5 0.7637626
#> 2     sp8 0.7637626
#> 3     sp2 0.5000000
```

8. Listing plots in clusters or combinations of clusters: `releves_in_clusters()`

8.1. Purpose

`releves_in_clusters()` returns the plot IDs (relevés) belonging to selected clusters based on `Plot.cluster > 0`. It supports both single clusters and combinations of clusters (OR across clusters).

If `values = TRUE`, it returns membership values instead of plot IDs:

- for a single cluster, the corresponding `Plot.cluster` values are returned;
- for a combination of clusters, the per-plot **maximum** `Plot.cluster` **value** across the selected clusters is returned (binary membership if `plot_values = "binary"`; maximum relative cover if `plot_values = "rel_cover"`).

If `drop0 = TRUE` (default), only plots belonging to the selected cluster(s) are returned. If `drop0 = FALSE`, non-member plots are also retained with membership value 0. This option is mainly useful when returning membership values rather than only plot IDs.

8.2. Usage

```
releves_in_clusters (
  x,
  clusters,
  values = FALSE,
  drop0 = TRUE,
  return = c("list", "vector", "matrix")
)
```

8.3. Examples

Single cluster:

```
releves_in_clusters(res, clusters = "c_2")
#> $c_2
#> [1] "plot2" "plot3" "plot7" "plot9"
```

Combination of clusters:

```
releves_in_clusters (
  res,
  clusters = list(group1 = c("c_2", "c_5"))
)
#> $g_2_5
#> [1] "plot2" "plot3" "plot4" "plot5" "plot6" "plot7"
#> [7] "plot8" "plot9" "plot10"
```

Membership values as a matrix:

```

releves_in_clusters (
  res,
  clusters = list(group1 = c("c_2", "c_5")),
  values    = TRUE,
  drop0     = FALSE,
  return    = "matrix"
)
#>           g_2_5
#> plot1  0.0000000
#> plot2  0.3548387
#> plot3  0.6060606
#> plot4  0.7222222
#> plot5  0.1666667
#> plot6  0.3333333
#> plot7  0.4166667
#> plot8  0.2608696
#> plot9  0.5833333
#> plot10 1.0000000

```

9. Finding clusters containing species: `clusters_with_species()`

9.1. Purpose

`clusters_with_species()` returns clusters whose cluster-constituting species set (from `Cluster.species`) contains one or more queried species. This is a fast, topological lookup (not a ϕ -based fidelity measure).

By default, it filters to clusters with `Cluster.height >= 0` via `min_phi = 0`.

9.2. Usage

```

clusters_with_species (
  x,
  species,
  match    = c("any", "all"),
  min_phi  = 0,
  return   = c("labels", "ids", "table")
)

```

9.3. Examples

Clusters containing a species:

```
clusters_with_species(res, species = "sp1")  
#> [1] "c_2" "c_3" "c_4" "c_6" "c_7"
```

Clusters containing *both* species, restricted to stronger clusters:

```
clusters_with_species(  
  res,  
  species = c("sp5", "sp8"),  
  match   = "all",  
  min_phi = 0.2  
)  
#> [1] "c_1"
```

10. Assigning plots (relevés) to candidate vegetation units: `assign_relevés()`

10.1. Purpose

`assign_relevés()` assigns each plot to one of a user-defined set of **candidate vegetation units** represented by one or several Cocktail clusters.

Candidate vegetation units are defined by `clusters`:

- If `clusters` is a **vector** (e.g. `c("c_12", "c_27")` or `c(12, 27)`), each element defines one candidate vegetation unit represented by a single cluster.
- If `clusters` is a **list**, each list element defines one candidate vegetation unit represented by several clusters (OR across cluster memberships).

Each plot is assigned by scoring all candidate vegetation units and selecting the best-scoring unit. Depending on `plot_membership`, scoring can be restricted to vegetation units for which the plot is already a Cocktail member (based on `Plot.cluster > 0`, combined across the clusters representing that unit), or allowed for all candidate vegetation units.

10.2. Interface

```

assign_relevés (
  x,
  strategy      = c("count", "cover", "phi", "phi_cover"),
  clusters,
  plot_membership = TRUE,
  min_phi       = NULL,
  min_group_size = 1L
)

```

10.3. Strategies

For each candidate vegetation unit u , `assign_relevés()` defines a set of candidate species S_u used for scoring.

"count"

- Score: number of species in S_u present in the plot.
- Candidate species: S_u is the cluster-constituting species set from `Cluster.species` (unioned if the vegetation unit is represented by several clusters).
- Ties set to "+". All zero set to NA.

"cover"

- Score: sum of cover values $\text{cover}(p, i)$ (from `vegmatrix`) of species in S_u present in the plot.
- Candidate species: S_u is the cluster-constituting species set from `Cluster.species` (unioned if the vegetation unit is represented by several clusters).
- Tie-breaker: species count; remaining ties set to "+". All zero set to NA.

"phi" (requires `Species.cluster.phi`)

- Define vegetation-unit-level $\phi(i, u)$ as the maximum ϕ across the clusters representing that unit; negative values are truncated to 0.
- Score: sum of $\phi(i, u)$ over species in S_u present in the plot.
- Tie-breakers: (1) total cover, (2) species count; remaining ties set to "+". All zero set to NA.
- Candidate species depend on `min_phi`:
 - `min_phi = NULL` (default): S_u contains only the cluster-constituting species.
 - numeric `min_phi` in $[0, 1]$: S_u contains species with $\phi(i, u) \geq \text{min_phi}$ (from the full ϕ profile).

"phi_cover" (requires `Species.cluster.phi`)

- Score: $\sum \text{cover}(p, i) \times \phi(i, u)$ over species in S_u present in the plot.
- Tie-breakers: (1) total cover, (2) species count; remaining ties set to "+". All zero set to NA.
- Candidate species depend on `min_phi`:
 - `min_phi = NULL` (default): S_u contains only the cluster-constituting species.
 - numeric `min_phi` in $[0, 1]$: S_u contains species with $\phi(i, u) \geq \text{min_phi}$ (from the full ϕ profile).

If "phi" or "phi_cover" is requested but `Species.cluster.phi` is missing, the function warns and falls back to "count" (if `plot_membership = TRUE`) or "cover" (if `plot_membership = FALSE`).

10.4. Example

```
# define candidate vegetation units from clusters at a phi cut
candidate_units <- clusters_at_cut(res, phi_cut = 0.3, as_labels = TRUE)

rel_count <- assign_relevés(
  x           = res,
  strategy    = "count",
  clusters    = candidate_units,
  plot_membership = TRUE,
  min_group_size = 1L
)

table(rel_count, useNA = "ifany")
#> rel_count
#>   u_1  u_3 <NA>
#>    5    3    2
```

ϕ -weighted assignment (requires `Species.cluster.phi`). When `plot_membership = FALSE`, all candidate vegetation units compete for every plot, regardless of whether the plot meets the Cocktail membership criterion (m-threshold) for any cluster representing that unit (i.e., regardless of `res$Plot.cluster > 0`):

```
rel_phi <- assign_relevés(
  x           = res,
  strategy    = "phi_cover",
  clusters    = candidate_units,
  plot_membership = FALSE,
  min_phi     = 0.25,
  min_group_size = 1L
)
```



```
table(rel_phi)
#> rel_phi
#> u_1 u_3
#>    4    5
```

11. Typical analysis workflow (dune dataset)

This section illustrates a typical workflow using the **dune** dataset included in the **vegan** package. The dataset comprises 20 vegetation plots and cover values for 30 plant species.

11.1. Step 1: Prepare vegetation matrix

```
library(vegan)
data(dune)

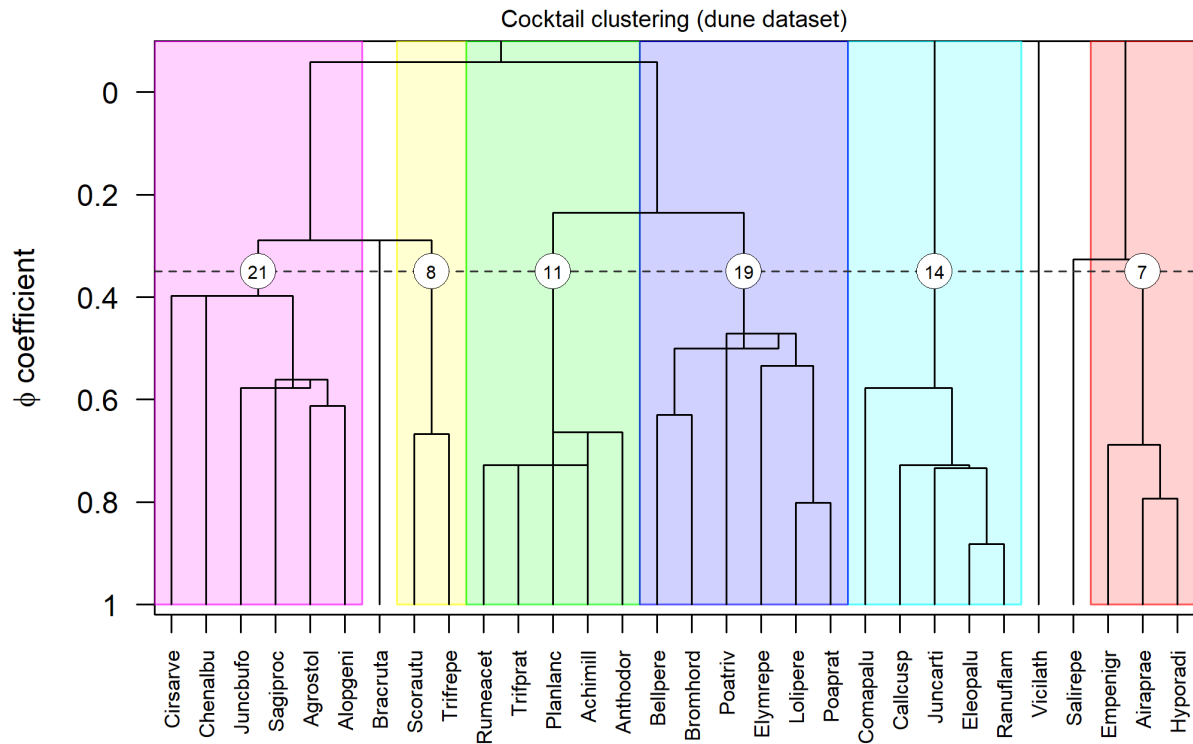
vm <- as.matrix(dune)
dim(vm)
#> [1] 20 30
```

11.2. Step 2: Run Cocktail clustering

```
res <- cocktail_cluster(
  vegmatrix      = vm,
  plot_values    = "rel_cover",
  species_cluster_phi = TRUE
)
#> |
```

11.3. Step 3: Plot the Cocktail dendrogram

```
cocktail_plot(
  x          = res,
  phi_cut    = 0.35,
  label_clusters = TRUE,
  cex_species = 0.7,
  main       = "Cocktail clustering (dune dataset)"
)
```



11.4. Step 4: Define candidate clusters

Parent clusters at a ϕ cut can be extracted with `clusters_at_cut()`:

```
clusters_parent <- clusters_at_cut(
  res,
  phi_cut = 0.35,
  as_labels = TRUE
)

clusters_parent
#> [1] "c_7" "c_8" "c_11" "c_14" "c_19" "c_21"
```

Candidate diagnostic species groups can be selected with `select_clusters()`:

```
clusters_strong <- select_clusters(
  x = res,
  min_phi = 0.20,
  min_k = 2L,
```

```

    min_n = 1L,
    min_score = 0,
    score_method = "h_logk_logm",
    mode = "strict",
    return = "labels"
)

clusters_strong
#> [1] "c_11" "c_19" "c_24" "c_14" "c_7"

```

11.5. Step 5: Species lists (cluster species and ϕ -ranked diagnostics)

Cluster-constituting species sets for the selected diagnostic groups:

```

diag_sp <- species_in_clusters(res, labels = clusters_strong)
str(diag_sp)
#> List of 5
#> $ c_11: chr [1:5] "Achimill" "Anthodor" "Planlanc" "Rumeacet" ...
#> $ c_19: chr [1:6] "Bellpere" "Bromhord" "Elymrepe" "Lolipere" ...
#> $ c_24: chr [1:9] "Agrostol" "Alopgeni" "Chenalbu" "Cirsarve" ...
#> $ c_14: chr [1:5] "Comapalu" "Eleopal" "Juncarti" "Ranuflam" ...
#> $ c_7 : chr [1:3] "Airaprae" "Empenigr" "Hyporadi"

```

ϕ -ranked diagnostic species lists for the selected diagnostic groups:

```

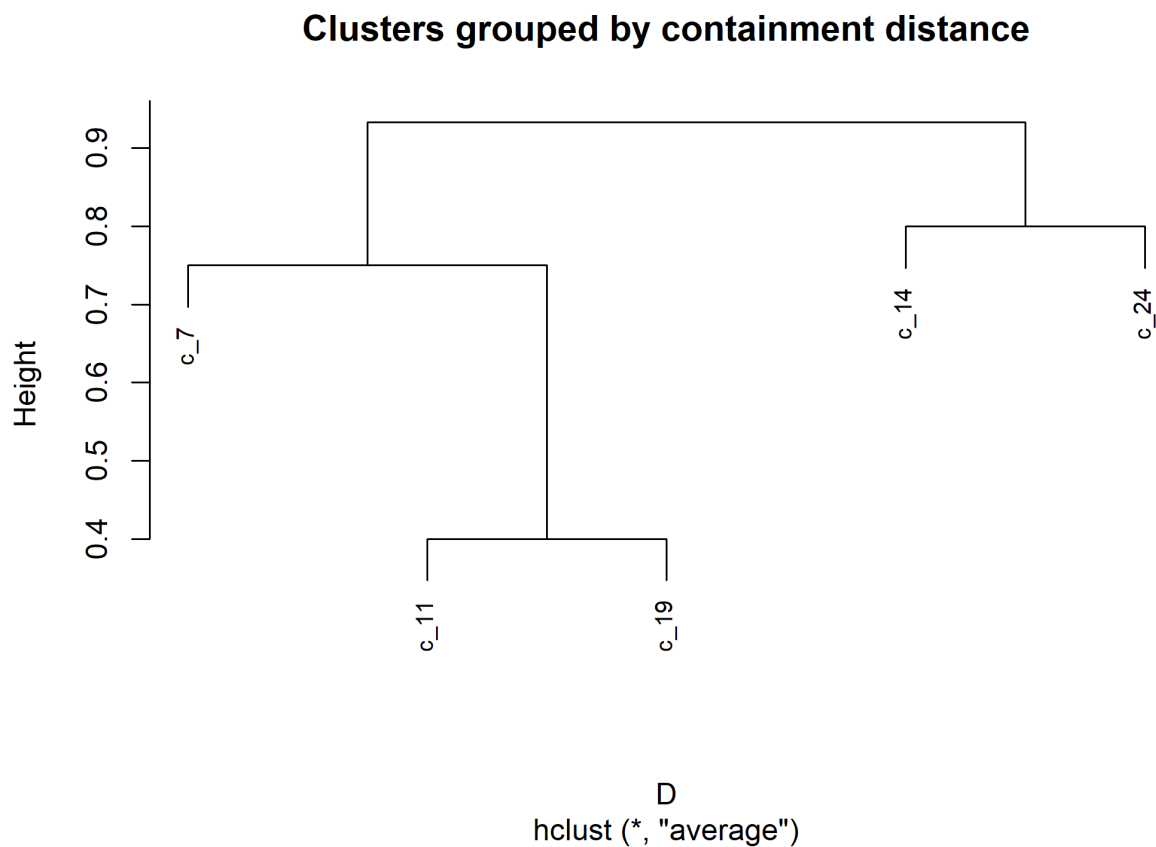
diag_sp_phi <- species_in_clusters(
  x = res,
  labels = clusters_strong,
  species_cluster_phi = TRUE,
  min_phi = 0.20
)

diag_sp_phi[[1]]
#>      species      phi
#> 1 Anthodor 0.8819171
#> 2 Achimill 0.7867958
#> 3 Planlanc 0.7867958
#> 4 Trifprat 0.7276069
#> 5 Bromhord 0.4666667
#> 6 Rumeacet 0.4666667
#> 7 Poaprat 0.3779645
#> 8 Lolipere 0.2357023

```

11.6. Step 6: Relationships among selected clusters

```
D <- cluster_dist(  
  res,  
  clusters = clusters_strong,  
  method = "containment"  
)  
  
hc <- hclust(D, method = "average")  
  
plot(  
  hc,  
  main = "Clusters grouped by containment distance",  
  cex = 0.8  
)
```



11.7. Step 7: Assign plots to candidate vegetation units

```
rel <- assign_relevés (
  x = res,
  strategy = "count",
  clusters = clusters_strong,
  plot_membership = TRUE,
  min_group_size = 1L
)

table(rel)
#> rel
#> u_11 u_14 u_19 u_24 u_7
#>    3    4    5    5    1
```

12. Reproducibility notes

- All ϕ computations in `cocktail_cluster()` and downstream functions are ϕ coefficients derived from 2×2 contingency tables (e.g. species presence × cluster membership).
- Deterministic ordering and tie-handling:
 - Species order is deterministic: species are kept in the original input column order after removing empty species (all-zero after binarization).
 - Clustering tie-handling is deterministic: when multiple candidate pairs share the same maximum association coefficient ϕ at a step, they are processed in a fixed order defined by the lower-triangular scan order (increasing column, then row).
 - Relevé (plot) assignment tie-handling is deterministic: in `assign_relevés()`, ties are resolved using strategy-specific secondary and, when applicable, tertiary criteria; if a tie remains, the plot is assigned "+" (explicit tie label).